

Creating Levels in AR FLOW

This guide describes the process of creating a new level for AR FLOW. It provides a step-by-step checklist and recommended practices. The guide assumes a laboratory-style level with an experimental setup located in a hall. Some instructions may not apply to fundamentally different level types (for example, the Potential Flow scene with a 3D plot, or the Reynolds Dye Experiment with a 2D dye filament).

The examples in this guide reference the `TutorialLevel` located in the `Assets/Scenes` directory. Use that scene as a template: copy it and adapt the copy for a new level.

Language

The project uses English for all game text, including level names and UI. Please keep new level content in English.

1 First Steps

Begin by familiarizing yourself with the project folder layout. The structure is intended to be self-descriptive. Follow the existing conventions when adding new assets. Typical contents of the `Assets` folder include:

- Images in `Images`
- 3D models in `Models`
- Code scripts in `Scripts`
- Other assets in their respective subfolders

After you finish this guide, review the existing levels to learn common patterns and practices. The project currently contains, among others, the following levels:

- **Potential Flow:** a 3D surface plot for visualizing potential flow fields.
- **3D Centrifugal Pump:** a 3D pump model with animated flow lines and measurement displays.
- **Pump Curve:** a virtual test rig for determining a pump's characteristic curve.
- **Pump Circuit:** a virtual test rig for measuring a washing machine's load characteristic.
- **Reynolds' Dye Experiment:** demonstration of laminar and turbulent flows using a dye filament.

- **Crossflow Valve:** visualization of flow and pressure distribution in a crossflow valve.

To show the `TutorialLevel` in the main menu for testing, enable the `showInMenu` flag in `Assets/Resources/Levels/TutorialLevel`. The flag is `false` by default (see fig. 1). For normal releases, leave this flag disabled. Use it only for temporary testing.

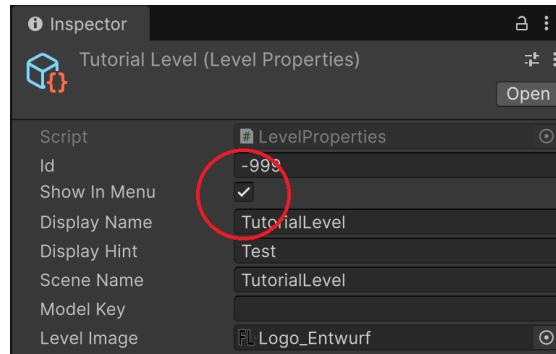


Figure 1: `LevelProperties` in the Inspector

2 Level Structure

A level is implemented as a Unity scene. In this project a scene normally corresponds directly to a playable level (exception: `ARScene`, see below). A typical scene is composed of two parts: the scene-specific `GameObjects` and a shared `Pipemaster_Base` prefab (see fig. 2).

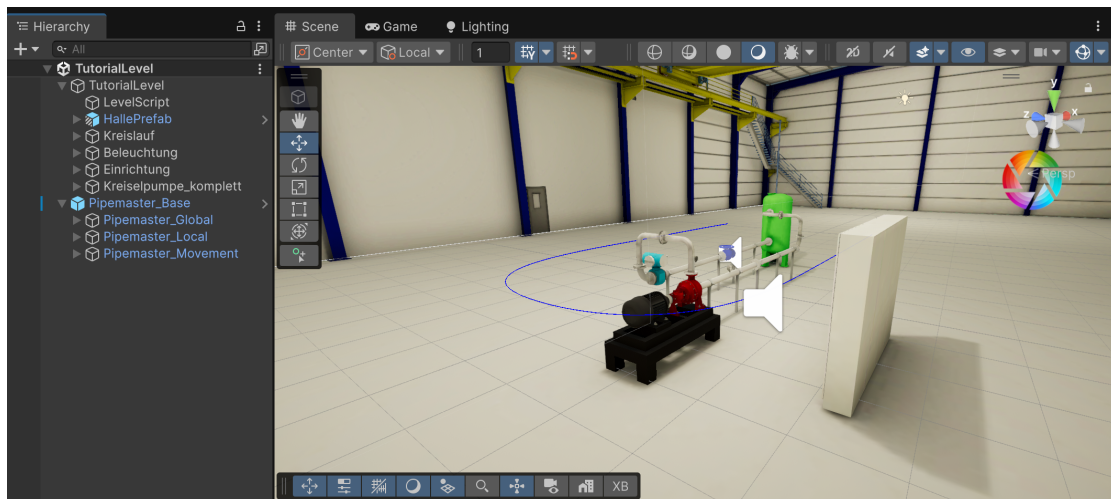


Figure 2: Level Structure

Exception: ARScene

The ARScene is not a standalone playable level. It is a generic AR container that receives a model identifier at runtime. Different levels can reuse the same ARScene while presenting different 3D models.

2.1 Unified Prefab

The `Pipemaster_Base` prefab centralizes components that should be consistent across levels. It exposes three child groups: `Pipemaster_Global`, `Pipemaster_Local`, and `Pipemaster_Movement`. Each group is documented below. For most levels it is sufficient to include the prefab without modification. Individual components may be disabled in a scene if they are not needed. Do not delete prefab assets.

Never overwrite the prefab!

Do not overwrite or delete the `Pipemaster_Base` prefab. Local changes are acceptable (for example, disabling a child `GameObject` in a scene), but applying changes to the prefab asset (e.g., using 'Apply All', see fig. 3) will propagate them to all levels and may introduce regressions.

2.1.1 Pipemaster_Global

`Pipemaster_Global` contains components that should persist for the application lifetime. These objects are protected with `DontDestroyOnLoad()` so they survive scene changes. Typically no per-level changes are required for this group.

2.1.2 Pipemaster_Local

`Pipemaster_Local` contains components that are instantiated for each scene. You can typically leave this group unchanged, but individual elements may be configured or disabled per level. The main subcomponents are:

- **EventSystem**: Unity built-in component, remains unchanged.
- **ToolbarUI**: Shows the toolbar and other UI elements that are similar in every level. Remains unchanged.
- **QuestManager**: Manages quests, remains unchanged.
- **InteractionMenuUI**: Displays the interaction menu for objects in the level, remains unchanged.
- **BackgroundAudio**: Plays background music. If needed, the audio clip in the `AudioSource` can be changed or the `GameObject` disabled entirely.

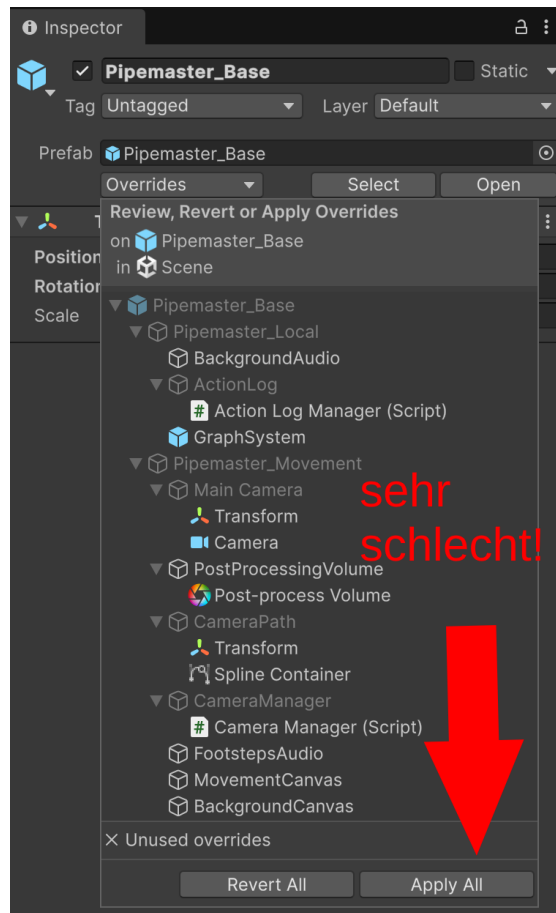


Figure 3: Never overwrite the prefab!

- **ActionLog:** Logs actions in the game, remains unchanged.
- **GraphSystem:** Provides the ability to display graphs. If the level does not need graphs, the entire GameObject can be deactivated.

2.1.3 Pipemaster Movement

Pipemaster_Movement groups components that handle player or camera movement in the scene. These components are instantiated per scene. The key elements are listed below.

- **Main Camera:** Unity built-in camera. In theory, camera settings (e.g., FOV) can be adjusted, but in practice we probably want consistent camera settings across levels.

- **PostProcessingVolume**: Post-processing effects for the camera. As with the camera itself: can be adjusted in theory, but in practice we want a consistent look across levels.
- **CameraPath**: Defines the spline path along which the camera travels. In most levels the camera follows a path, so this component should usually not be disabled. The path, however, must be adapted to each level.
- **CameraManager**: Manages camera logic and switching between different camera modes. In the subcomponents you can adjust the individual camera modes. Below are only the subcomponents that require relevant adjustments.
 - **CameraManager**: Initial camera mode (**camType**), usually **SplinePath** in lab-based levels.
 - **SplineCameraMode**: Initial position (**position**) of the camera on the path defined in **CameraPath**.
 - **AerialCameraMode**: Rotation of the camera in bird’s-eye view, usually tilted slightly down (**cameraRotation**), and the imaginary rectangle in the scene (**bounds**) the camera is allowed to move within. We don’t want the player to escape the hall in bird’s-eye view.
- **FootstepsAudio**: Plays footstep sounds when moving the camera. If needed, the audio clip in the **AudioSource** can be changed or the **GameObject** disabled entirely.
- **MovementCanvas**: Shows the joystick, remains unchanged.
- **BackgroundCanvas**: Shows the background image for the detail view, remains unchanged.

2.2 Scene-specific Components

Scene-specific components are the **GameObjects** that are unique to a scene and not part of the shared prefab. By convention these are parented under a root **GameObject** named after the scene (for example, 'TutorialLevel' in fig. 2). This yields a consistent top-level layout across levels and simplifies navigation.

The exact set of scene-specific objects varies between levels. When creating a new lab-based level from **TutorialLevel**, it is common to reuse the hall geometry and lighting and replace or extend the experimental apparatus. Every level normally includes one **LevelScript** that implements the level logic.

2.2.1 LevelScript

Each level provides a `LevelScript` that implements its game logic. By convention these scripts live in `Scripts/LevelScripts`. Typical responsibilities of a `LevelScript` include:

- Registering the level's `QuestLine` with the `QuestManager` so quests are available in the UI. See section 2.3.1 for details.
- Configuring the `ToolbarManager` to select which UI buttons and panels are visible. This affects:
 - Buttons in the main toolbar at the bottom right.
 - `LevelObjects` buttons at the top left.
 - `FieldProperties` at the top right, used to display various measured values, e.g., in the '3D Centrifugal Pump' level.
 - Camera button at the top right to switch between camera modes in lab-based levels.

Details about the various UI elements follow in section 2.3.2.

- Storing references to relevant `InteractionObjects` and computing the physical relationships between them. See the lab-based `LevelScripts` (e.g., `Pumpenkennlinie` and `ReihenParallelSchaltung`) for concrete examples. Detailed information on `InteractionObjects` follow in section 2.2.2.

In general, the `LevelScript` of the `TutorialLevel` is very simple because the level has little functionality. After finishing the guide it is therefore helpful to look at the `LevelScripts` of the other levels. An overview of all levels can be found in section 1.

2.2.2 InteractionObjects

`InteractionObjects` are scene objects representing components of the experimental setup that the player can interact with. Clicking an `InteractionObject` opens its `InteractionMenu` (see fig. 4). Each `InteractionObject` defines which UI elements its `InteractionMenu` displays. For example, the electric motor appears in multiple levels such as 'Pump Curve' and 'Pump Circuit'.

The `InteractionObject` for the electric motor is implemented in the `MotorObject` class inside `InteractionObjects`. All `InteractionObjects` inherit from the `InteractionObject` base class, which provides common fields that must be set in the inspector for each instance (see fig. 5):

- **name:** Display name in the UI



Figure 4: InteractionMenu of the electric motor

- **questObject**: QuestObject associated with this InteractionObject. Details follow in section 2.3.1.

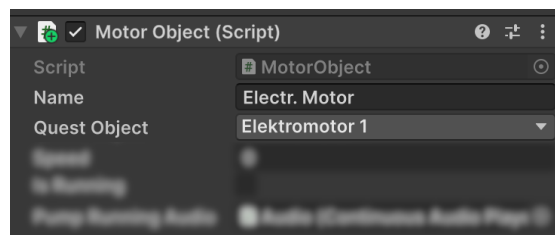


Figure 5: MotorObject in the Inspector

Subclasses implement object-specific behavior and add the appropriate menu entries (for example by calling `AddMenuElement()`). The insertion order determines the display order. Reusable menu elements are defined under `InteractionMenus/MenuElements`.

Conventions for InteractionMenus

By convention, all InteractionObjects show a button as the first element in the InteractionMenu that opens the detail view for this object. Therefore, in all subclasses of `InteractionObject` you will likely find at the beginning a call like `AddMenuElement(new Button("Detail View", OpenDetailView))`.

Each InteractionObject maintains its own state. For the electric motor this includes the set speed and whether it is powered. LevelScripts often query these state variables to compute system behavior.

InteractionObjects commonly register with the `QuestManager` to react to quest events.

For example, a quest may disable certain menu options. In the motor example a quest can disable the speed slider until a later quest re-enables it. See section 2.3.1 for the quest interaction model.

2.2.3 Lighting

Every hall needs lighting. For example, the TutorialLevel has three light sources under *Beleuchtung*. In general there are two ways lighting can work in a level:

- **Static**, i.e., the lighting is precomputed ("baked") and then fixed in the scene.
- **Dynamic**, i.e., the lighting is calculated in real time.

During development of a level it is useful to set all lights to dynamic (**Mode -> Realtime**, see fig. 6). That way a level can be tested immediately without rebaking the lighting after geometry changes. Otherwise you'd have to rebake the lighting after each change in the scene geometry, which on my laptop takes about 10 minutes depending on the scene.

Set lighting to dynamic while developing!

For developing and testing the level, set lighting on all lights to dynamic (**Mode -> Realtime**), and only switch back to static once the level is finished.

When the level is finished, lighting can be switched back to static (**Mode -> Baked**) so the level runs smoothly on phones. Generating the lightmaps correctly, however, requires additional steps described in section 3.2.

2.3 Further Concepts

This section introduces further concepts relevant to level development that do not directly exist as *GameObjects* in the scene.

2.3.1 Quests

Quests are tasks the player should complete in the level. They guide the player through the level and help achieve the learning goals. Quests are defined in a nested way, i.e.:

- A **Quest** represents a single task, e.g., "Turn on the pump".
- A **QuestStage** is a list of **Quests** that must be completed to finish the **QuestStage** and unlock the next **QuestStage**. Within a **QuestStage** quests can be completed in any order.

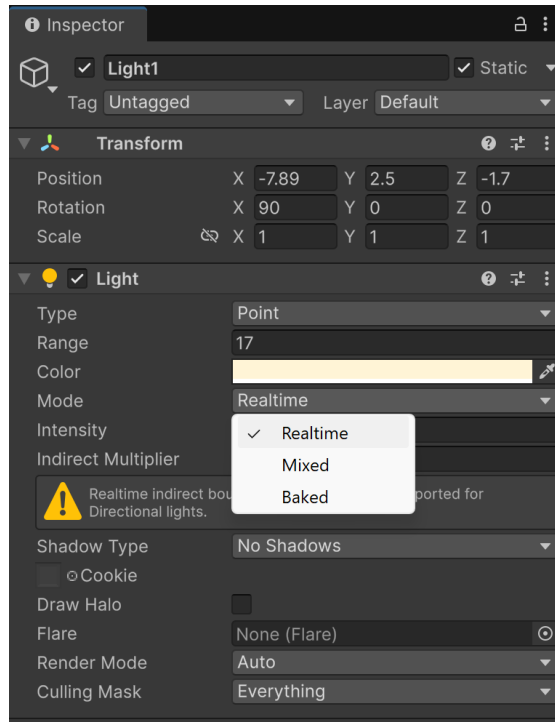


Figure 6: Set lighting to dynamic

- A **QuestLine** is a list of **QuestStages** that are unlocked sequentially. When a **QuestStage** is completed, the next **QuestStage** is unlocked. A **QuestLine** thus defines the entire quest structure of a level, so 1 level = 1 **QuestLine**.

A particularity of the quest system is that there is mainly only one type of communication between the quest system and the level itself (**LevelScript**, **InteractionObjects**, etc.). That communication happens through **QuestInteractions** and works bidirectionally:

- Objects in the level send a **QuestInteraction** to the **QuestManager** to inform the quest system that a certain action has been performed.

Example: The electric motor sends a **QuestInteraction** when its speed is changed. The **QuestManager** then automatically checks whether this action is relevant for any quest and updates quest progress accordingly. "Relevant" here means that a quest contains this interaction exactly as its **QuestTarget**.

- **Quests** and **QuestStages** broadcast a **QuestInteraction** on certain state changes (e.g., when a quest is completed). Interested objects in the level can register with the **QuestManager** to receive these **QuestInteractions** and react accordingly.

Example: When the quest "Set speed to 1000 RPM" is completed, the quest broadcasts a **QuestInteraction** named **BlockMotorSpeed**. The electric motor receives

this `QuestInteraction` and disables the speed slider until another quest broadcasts a different `QuestInteraction` that unlocks it again.

A `QuestInteraction` always consists of three components:

- The `QuestObject` the interaction is linked to. In most cases this is a real object in the scene, e.g., the electric motor. For each object that interacts with the quest system, there should be a unique `QuestObject` defined within the level (e.g., `Elektromotor1` and `Elektromotor2`).
- The `QuestInteractionType` defines the type of interaction, i.e., the "what". Examples: `SetMotorSpeed` or `BlockMotorSpeed`. Depending on the interaction type additional data may be relevant, e.g., the set speed for `SetMotorSpeed`. The generic type of the `QuestInteractionType` thus defines the type of the argument for the interaction, e.g., `float` for the speed. Some interaction types, like `BlockMotorSpeed`, do not need an argument. In that case the generic type is `object` (`object = none`).
- The value of the interaction, i.e., the argument (e.g., the speed for `SetMotorSpeed`). It must match the type defined by the `QuestInteractionType`, e.g., `float` for `SetMotorSpeed`. If the interaction type requires no argument (`object`), this field is left empty.

Both `QuestObject` and `QuestInteractionType` are used similarly to enums. Each class has a predefined list of possible values. All possible values for `QuestObject` live in `QuestObjects`, and `QuestInteractionType` values live in `QuestInteractionTypes`. This approach may look complex at first, but it provides type guarantees and eliminates the risk of typos in names or interaction kinds. It's worth it, especially for a growing project!

Always use `QuestInteractions`!

In theory the `QuestManager` offers callbacks like `onQuestCompleted` that could be used to react to quest completion. Then an object in the level would need to check the exact quest name to know when to react. This is error-prone (typos in quest names) and inflexible.

Therefore, even if the other approach seems simpler at first, always use `QuestInteractions` (`QuestManager.onInteractionFromQuest`) to react to quest state changes!

After explaining the quest system theoretically, the following two steps should be performed to implement quests in a new level:

1. Create a `QuestLine`.
2. Register and properly handle callbacks in the `QuestManager`, especially to handle quest interactions correctly.

A `QuestLine` is by convention defined as a static variable in the static class `QuestLines`. Each instance of `QuestLine` maps exactly to the quest structure described here. A quick skim of existing `QuestLines` should be enough to understand the syntax. A few extra notes:

- Name and Description of a `QuestLine` are the texts shown in the LevelInfo box (i button at the bottom right of the toolbar, see fig. 7).
- The Description of a `QuestStage` can be left empty because it is currently not displayed in the UI. We may find a place for it later, but for now it is unused.

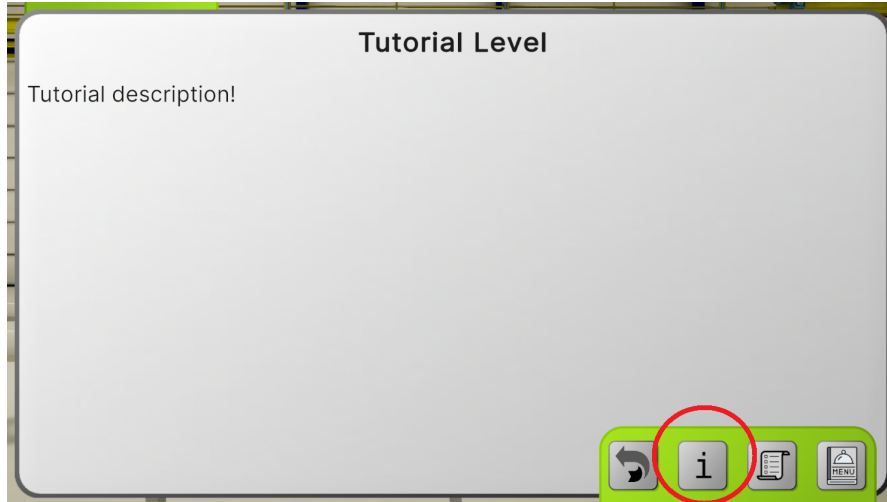
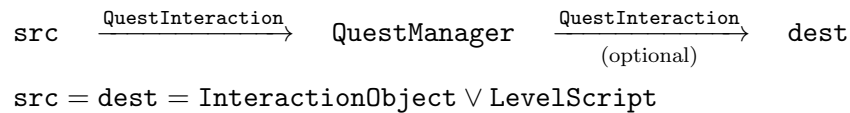


Figure 7: LevelInfo box

To register the completed `QuestLine` in the level scene, pass it to the `QuestManager` in the level's `LevelScript`. For the `TutorialLevel` this happens with the function call `SetQuestLine(QuestLines.TutorialLevel)`. You will find such a line in every `LevelScript` because each level has its own `QuestLine`. Even if a level does not use quests, an empty `QuestLine` must still be defined and passed so the LevelInfo box is filled correctly.

Finally, react to the `QuestManager` callbacks if there are quests that themselves broadcast a `QuestInteraction` in response. Recall the bidirectional communication:



When a quest broadcasts a `QuestInteraction` in response, where to react depends on the context and type of interaction. In practice both are possible: in the `LevelScript` or in a specific `InteractionObject`. `QuestManager.onInteractionFromQuest` is the

callback through which the `QuestInteraction` is received in both cases. An example can be found in the TutorialLevel's `LevelScript`.

Use `IQuestInteraction.IsObjectAndType` to check which interaction was received. `IsObjectAndType` has two overloads: one for interactions without parameters (e.g., `UseDetailView`), and one for interactions with parameters (e.g., `SetMotorSpeed`). In the first case only `QuestObject` and `QuestInteractionType` are checked. In the second case the value of the interaction parameter is also returned as an `out` parameter.

2.3.2 UI Elements

The `ToolbarManager` manages a variety of reusable UI elements that are used across different levels. This includes more than just the main toolbar at the bottom right, as shown in fig. 8. Below are the main UI elements used in most levels, with a short explanation.

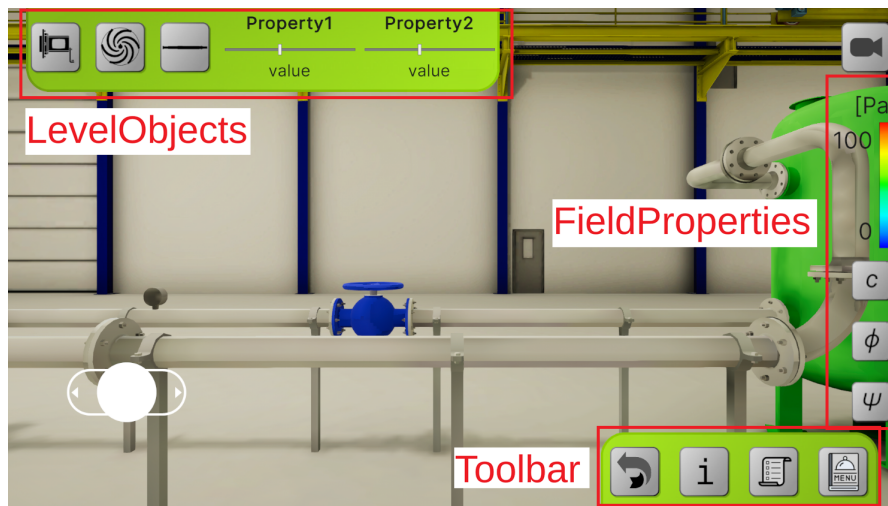


Figure 8: Reusable UI elements within a level

- **Toolbar:** Main toolbar at the bottom right with buttons whose functions should be consistent across levels.
- **LevelObjects:** Buttons at the top left that can be assigned individually per level.
- **FieldProperties:** Colored scale with additional buttons, used as a legend for measured values.
- Small helpers like a **HintBox** for tips (`ShowHintWithText`) or a **FunctionBox** to show functions in the `Potential Flow` level.

The `ToolbarManager` provides unified methods to control all these UI elements. Together with the various enums and helper classes in the same namespace (`ToolbarButton`, `ToolbarArea`, etc.) all UI elements in any level can be controlled in a simple and consistent way. Exception: the `Property1` and `Property2` sliders are public variables in the `ToolbarManager` because they are currently used in only one level and no worthwhile abstraction exists yet.

In general, a look at the class code and some examples in the LevelScripts should be enough to understand usage. For example, looking at the enums mentioned earlier you will see that there is an enum value for almost every button and UI element. Which buttons or UI elements to customize in a level is a matter of judgment. For instance, you probably do not want to change the function of the MainMenu button or hide the toolbar entirely.

2.3.3 Action Log

The Action Log records relevant player actions. It displays a short message with an emoji in the top-left corner of the screen that disappears after 8 seconds (see fig. 9). This gives the player helpful feedback about their actions, e.g., whether an interaction succeeded or a quest was completed. If multiple messages are sent in succession the player can scroll through them like a list.

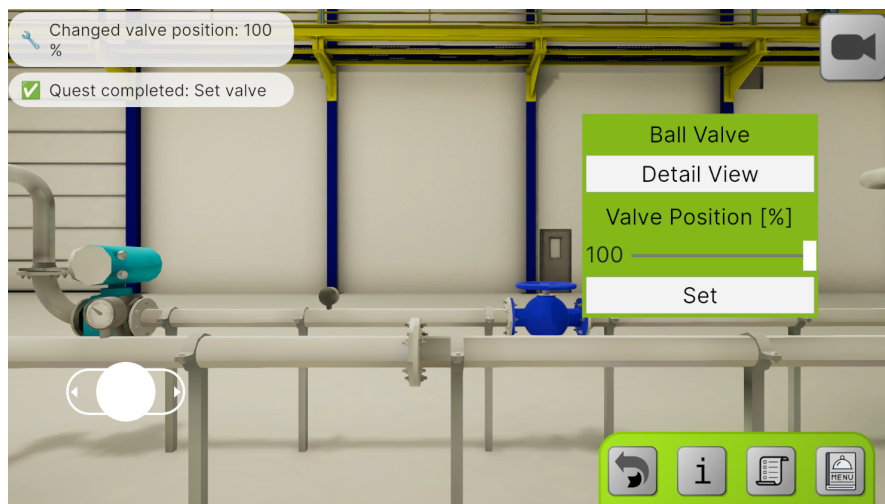


Figure 9: Example messages in the Action Log

To simplify its usage, the `ActionLogManager` exposes only a single public method: `Write(LogMessage message)`. `LogMessage` is a simple `struct` that stores two strings: one for the emoji and one for the message text. By convention all messages used across the game are defined centrally in the `LogMessages` class. If a message expects dynamic parameters it is implemented as a static method. Otherwise it is a `static readonly`

variable. This simplifies reuse and ensures consistent wording and emojis across the game.

When the Action Log was originally implemented there were issues with emoji rendering on Android devices. To fix this, the `NotoColorEmoji` font was added in `Assets/Fonts` so emojis render correctly. Since then emojis should display on Android without problems, but still:

Test emojis in the Action Log!

If a new message in `LogMessages` uses a completely new emoji not used anywhere else, test whether the emoji renders correctly on Android devices!

3 Embedding the Level

The scene for the new level is complete. The remaining integration steps are: register the level in the main menu, and bake static lighting if the level will target mobile devices. See section 3.1 and section 3.2 for details.

3.1 Registering in the Main Menu

Create a `LevelProperties` asset under `Assets/Resources/Levels` to make the level available in the main menu. The `MainMenuManager` enumerates all assets in that folder and builds the menu automatically. The example asset for `TutorialLevel` is shown in fig. 1. Key fields are:

- `id`: Numerical identifier used for sorting. Lower IDs appear earlier in the list.
- `showInMenu`: If true, the level is listed in the main menu.
- `displayName`: The name shown in the menu.
- `displayHint`: A short hint displayed above the name (for example, "Level 1").
- `sceneName`: The Unity scene name associated with the level.
- `modelKey`: Identifier for an AR model loaded by the `ARScene`. It is only relevant for `ARScene` entries.
- `levelImage`: Preview image shown in the menu.

3.2 Static Lighting

Unity supports static (baked) and dynamic lighting. Dynamic lighting requires no additional setup and is computed every frame. However, in practice dynamic lighting severely impacts performance on mobile devices. On our reference hardware (Samsung Galaxy A53 5G) switching scenes from dynamic to baked lighting produced roughly a 3–4x increase in framerate for complex lab scenes (e.g., **Pump Circuit**: 15 fps dynamic vs. 40–60 fps baked).

Static lighting is therefore recommended for complex lab-based levels destined for mobile platforms. Simpler levels that contain a single 3D model (**Potential Flow**) or only 2D content may run acceptably with dynamic lighting. Test performance before choosing a lighting strategy.

Implementing baked lighting involves two phases: preparing the scene and running the bake.

3.2.1 Preparing the Scene

Perform the following preparation steps:

- Set all scene lights to **Baked** and configure shadow types.
- Mark static geometry as **Static**.

Set primary shadow-casting lights appropriately. In the TutorialLevel only **Light2** uses **Soft Shadows**, while other lights use **No Shadows** to avoid cluttered shadowing (see fig. 10). If the new level uses the same hall geometry as the TutorialLevel, reuse its lighting settings.

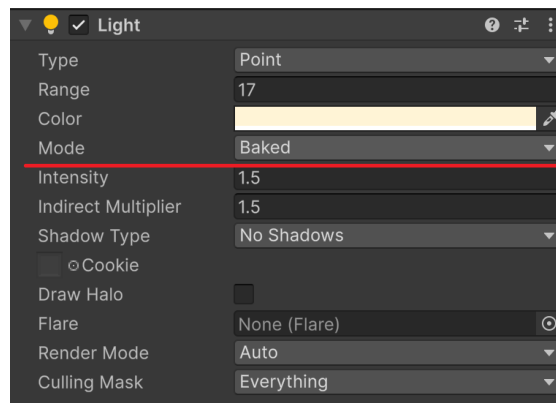


Figure 10: Switch lighting to static

Mark static meshes with the **Static** flag in the Inspector (see fig. 12). For complex pre-fabs, include child objects where appropriate. In the **MeshRenderer** verify the **Lighting**

settings and **Global Illumination** options. After a successful bake the Inspector shows the associated **Baked Lightmap** entry for each mesh.

In the model import settings (not in the **MeshRenderer**) set **Normals** to **Calculate** and enable **Generate Lightmap UVs** to improve bake quality (see fig. 11).

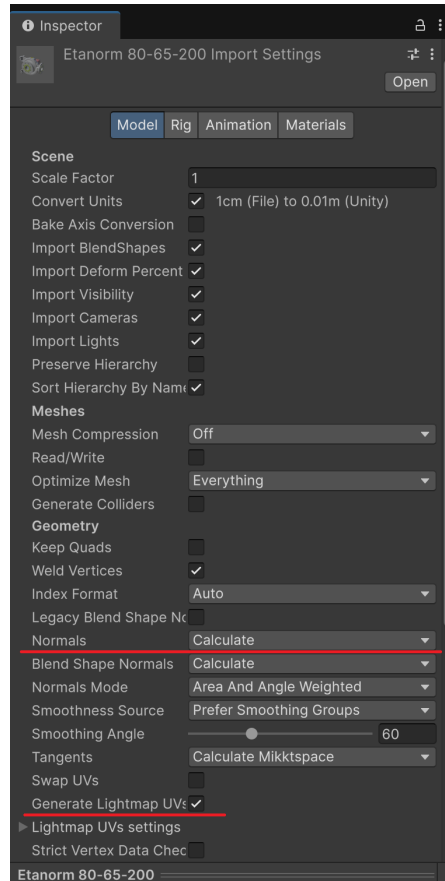


Figure 11: Inspector of a 3D model

3.2.2 All static? The consequences of static lighting

Baked lighting implies that lighting and shadowing for marked geometry are fixed in the generated lightmaps. Consequences:

- Shadows from static objects are immutable.
- The per-vertex/per-pixel shading baked into the lightmap does not update with object movement.

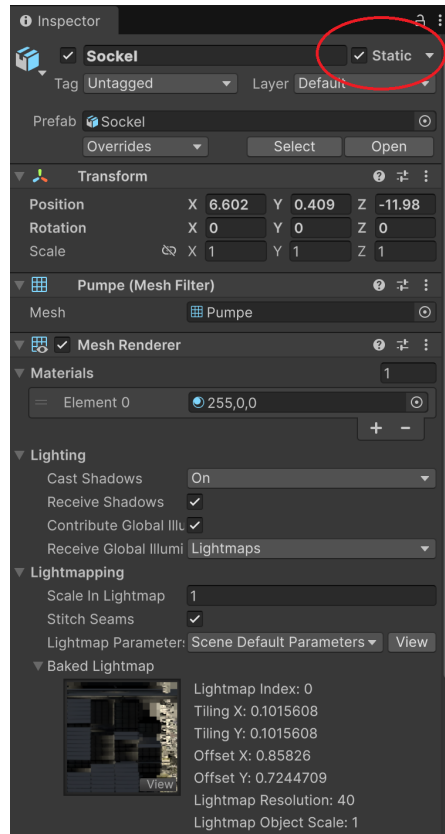


Figure 12: Set models to **Static**

Avoid moving or animating objects marked as **Static**. Small local animations can be acceptable if you disable **Batching Static** on the animated child so it remains addressable by the animator. For example, the valve wheel in the pump scene can remain visually acceptable when animated if you exclude it from batching (see fig. 13). For significant movement or large animated elements consider **Mixed** lighting instead.

Static objects must remain (almost) static!

Objects marked as **Static** should not be moved or undergo large animations. For substantial motion use a mixed lighting configuration.

3.2.3 Baking the Lighting

Open the Unity menu **Window > Rendering > Lighting** and select the **Scene** tab. Press **Generate Lighting** to start the bake (see fig. 14). Default settings are tuned for a balance between quality and compute time. Adjust only if you understand the trade-offs.

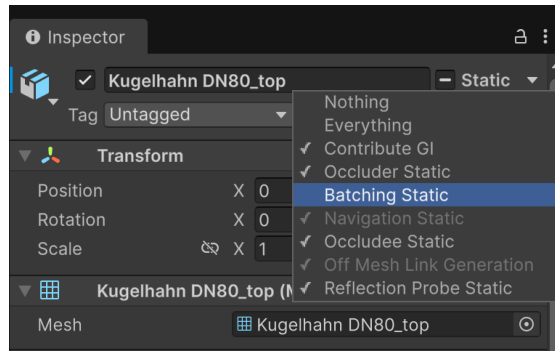


Figure 13: Detailed **Static** settings

The bake time depends on scene complexity and hardware. On the author's machine it can take several minutes up to about 10 minutes. After the bake completes, verify that each static mesh shows a **Baked Lightmap** entry in its Inspector (see fig. 12).

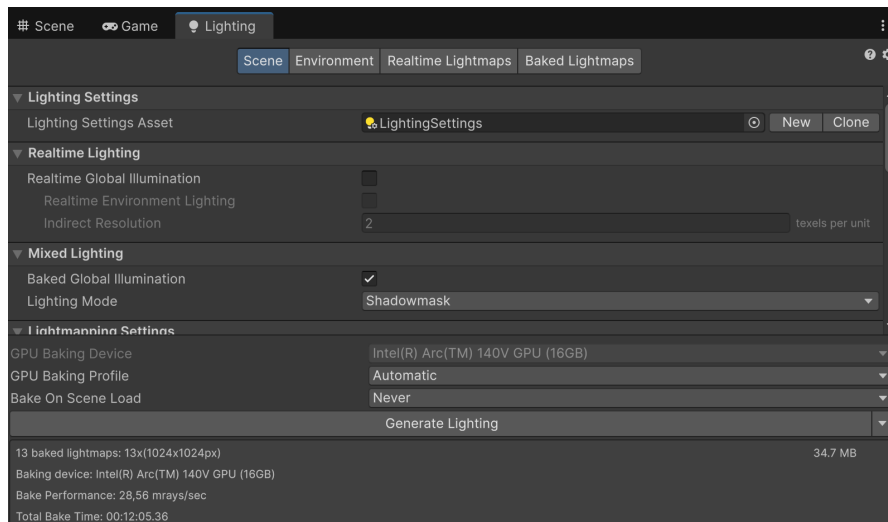


Figure 14: **Lighting** window to compute static lighting

Scene changed? Re-bake!

After modifying scene geometry or object placement, re-run the bake to update lightmaps. During major edits keep lighting dynamic for faster iteration.

Finally, confirm that no meshes show the "too large for lightmap" warning in the Inspector. If Unity flags a mesh as too large, reduce its **Lightmap Scale** in the **MeshRenderer** until the warning disappears. Conversely, increase **Lightmap Scale** for meshes that appear blotchy to improve quality.

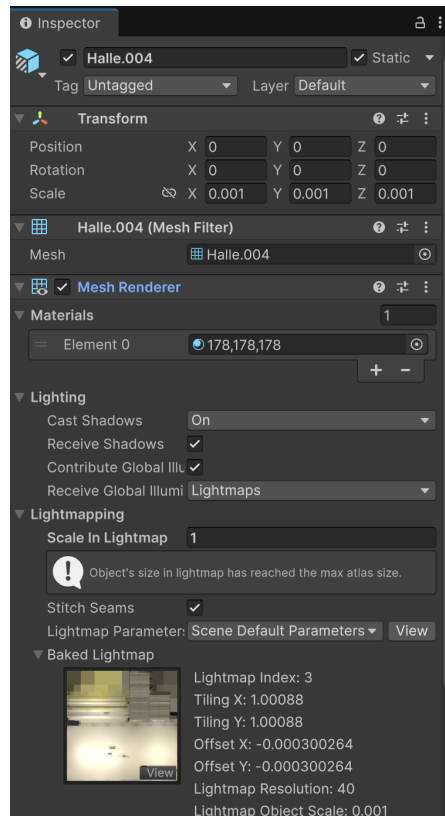


Figure 15: Model too large for a lightmap